Malware Analysis

Monnappa K A, Learning Malware Analysis, Packt Publishing Ltd, 2018..

Shubha Malige

Asst.Professor
CSE(Cyber Security)

Unit II

Inspecting PE Header Information, Comparing and Classifying The malware. **Dynamic Analysis:** System And Network Monitoring, Dynamic Analysis (Monitoring) Tools, Dynamic Analysis Steps, Putting it All Together: Analyzing a Malware Executable, Dynamic Link Library (DLL) Analysis.

- Pedagogy: Chalk and board, Problem based learning, Demonstration.

Dynamic Analysis-System and Network Monitoring

Dynamic Analysis

Dynamic analysis (behavioral analysis) involves analyzing a sample by executing it in an isolated environment and monitoring its activities, interaction, and effect on the system

System and Network Monitoring

During dynamic analysis, when the malware is executed, you will carry out various monitoring activities. The objective is to gather real-time data related to malware behavior and its the impact on the system. The following list outlines different types of monitoring carried out during dynamic analysis:

- **Process monitoring**: Involves monitoring the process activity and examining the properties of the result process during malware execution.
- **File system monitoring**: Includes monitoring the real-time file system activity during malware execution.
- **Registry monitoring**: Involves monitoring the registry keys accessed/modified and registry data that is being read/written by the malicious binary.
- **Network monitoring**: Involves monitoring the live traffic to and from the system during malware execution.

Dynamic Analysis(Monitoring) Tools-

- ✔ Before performing dynamic analysis, it is important to first understand what dynamic analysis means and what tools are required. Dynamic analysis involves running a suspected malware sample in a controlled virtual environment to observe its real-time behavior, so proper setup of tools inside a virtual machine is necessary.

- ✔ The required monitoring tools must be installed in the virtual machine and executed with administrator privileges, because malware often interacts with system files, registry, processes, and network components. Running tools as administrator ensures they can fully monitor and capture the malware's activities while it is executing.

Dynamic Analysis

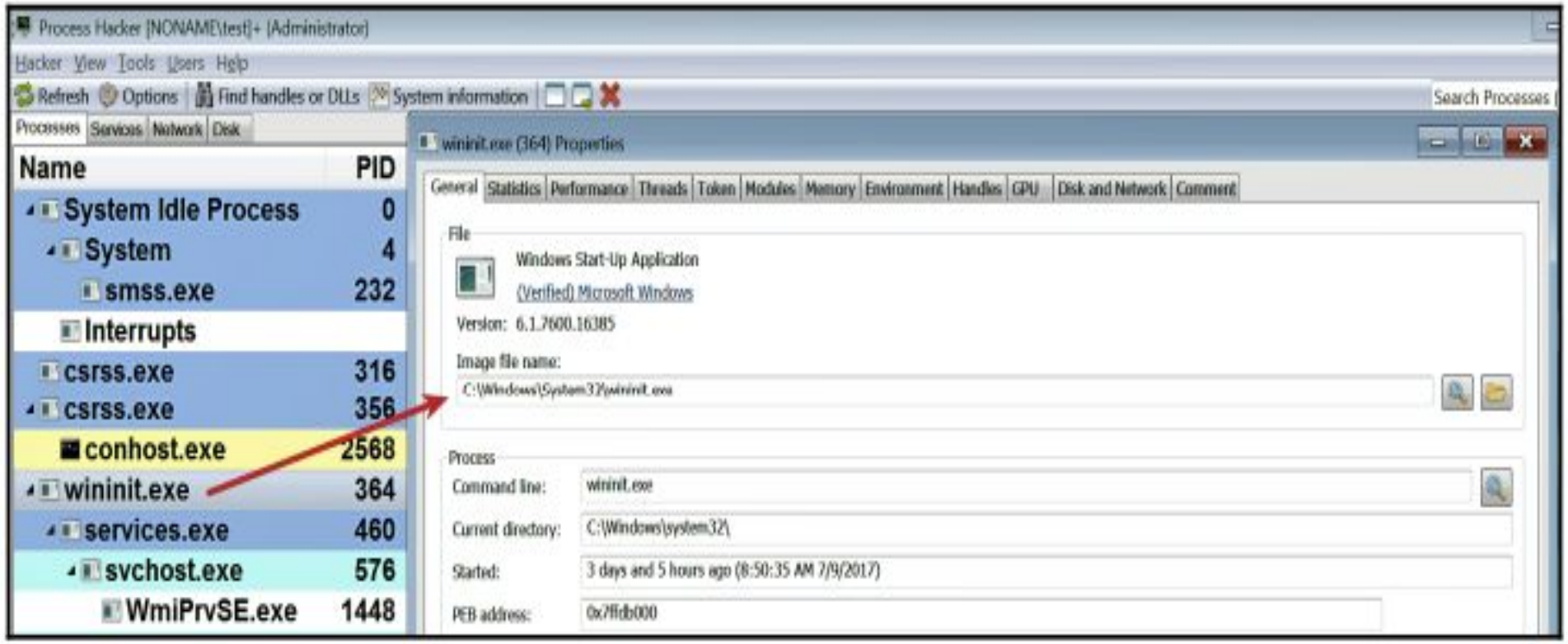
Dynamic Analysis Monitoring Tools:- Process Inspection with Process Hacker

- ✔ Process Hacker is an open-source, multipurpose tool that helps monitor running processes and system resources. It allows analysts to inspect process attributes, explore services, network connections, and disk activity, providing a detailed view of the system.

- ✔ Malware process identification and control: When a malware sample is executed, Process Hacker can help identify the newly created malware process, including its name and process ID. Analysts can examine various properties of the process and, if needed, terminate it to prevent further malicious activity.

Process Inspection with Process Hacker

The following screenshot shows Process Hacker listing all the processes running on the system, and the properties of `wininit.exe`:



Dynamic Analysis Tools

Determining System Interaction with Process Monitor

- ✔ Real-time monitoring of system activity: Process Monitor is an advanced tool that tracks and displays the real-time interactions of processes with the filesystem, registry, and process/thread activities, helping analysts observe how a program behaves on the system.

- ✔ Capturing and managing events: When run as Administrator, Process Monitor captures all system events automatically. Analysts can pause the event capture using Ctrl + E or clear all recorded events using Ctrl + X to focus on specific activities.

Determining System Interaction with process monitor

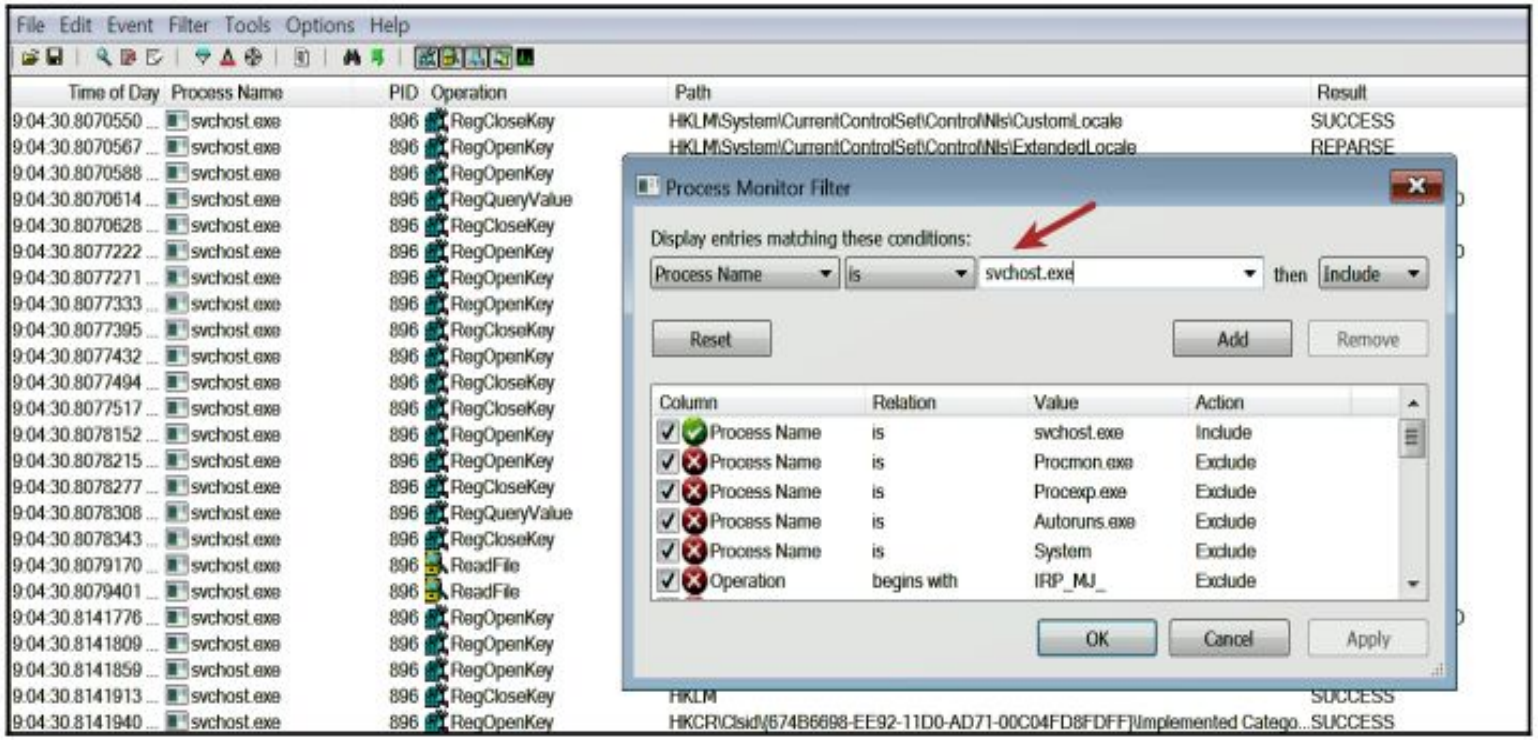
File Edit Event Filter Tools Options Help

Time of Day	Process Name	PID	Operation	Path
9:04:30.7830867 ...	wmiprvse.exe	2660	RegCloseKey	HKCR\CLSID\{D2D588B5-D081-11D0-99E0-00C04FC2F8EC}
9:04:30.7831292 ...	svchost.exe	896	RegOpenKey	HKU\S-1-5-18_Classes
9:04:30.7831323 ...	svchost.exe	896	RegOpenKey	HKLM
9:04:30.7831359 ...	svchost.exe	896	RegOpenKey	HKCR
9:04:30.7831395 ...	svchost.exe	896	RegCloseKey	HKLM
9:04:30.7831418 ...	svchost.exe	896	RegOpenKey	HKCR\Clsid\{674B6698-EE92-11D0-AD71-00C04FD8FDFF}\Implemented Catego...
9:04:30.7831450 ...	svchost.exe	896	RegCloseKey	HKCR
9:04:30.7831466 ...	svchost.exe	896	RegCloseKey	HKCR\CLSID\{674B6698-EE92-11D0-AD71-00C04FD8FDFF}\Implemented Cate...
9:04:30.7916790 ...	lsass.exe	468	RegOpenKey	HKLM\SAM\SAM\DOMAINS\Account\Groups\000003E8
9:04:30.7916869 ...	lsass.exe	468	RegOpenKey	HKLM\SAM\SAM\DOMAINS\Account\Aliases\000003E8
9:04:30.7916911 ...	lsass.exe	468	RegOpenKey	HKLM\SAM\SAM\DOMAINS\Account\Users\000003E8
9:04:30.7916979 ...	lsass.exe	468	RegQueryValue	HKLM\SAM\SAM\Domains\Account\Users\000003E8\V
9:04:30.7917022 ...	lsass.exe	468	RegCloseKey	HKLM\SAM\SAM\Domains\Account\Users\000003E8
9:04:30.7919684 ...	wmiprvse.exe	2660	CreateFile	C:\Windows\System32\wbem\wbemprox.dll
9:04:30.7920198 ...	wmiprvse.exe	2660	QueryBasicInformationFile	C:\Windows\System32\wbem\wbemprox.dll
9:04:30.7920226 ...	wmiprvse.exe	2660	CloseFile	C:\Windows\System32\wbem\wbemprox.dll
9:04:30.7920765 ...	wmiprvse.exe	2660	CreateFile	C:\Windows\System32\wbem\wbemprox.dll
9:04:30.7921268 ...	wmiprvse.exe	2660	CreateFileMapping	C:\Windows\System32\wbem\wbemprox.dll
9:04:30.7921381 ...	wmiprvse.exe	2660	CreateFileMapping	C:\Windows\System32\wbem\wbemprox.dll

Determining System Interaction with process monitor

- ✔ Process Monitor captures a lot of activity even on a clean system, making it difficult to focus on malware behavior.
- ✔ You can use the filtering feature to hide unwanted entries and display only the events you are interested in.
- ✔ Example of filtering: By setting a filter (Filter menu → Filter or Ctrl + L), you can focus on a specific process, such as svchost.exe, to analyze its activity.

Determining System Interaction with process monitor

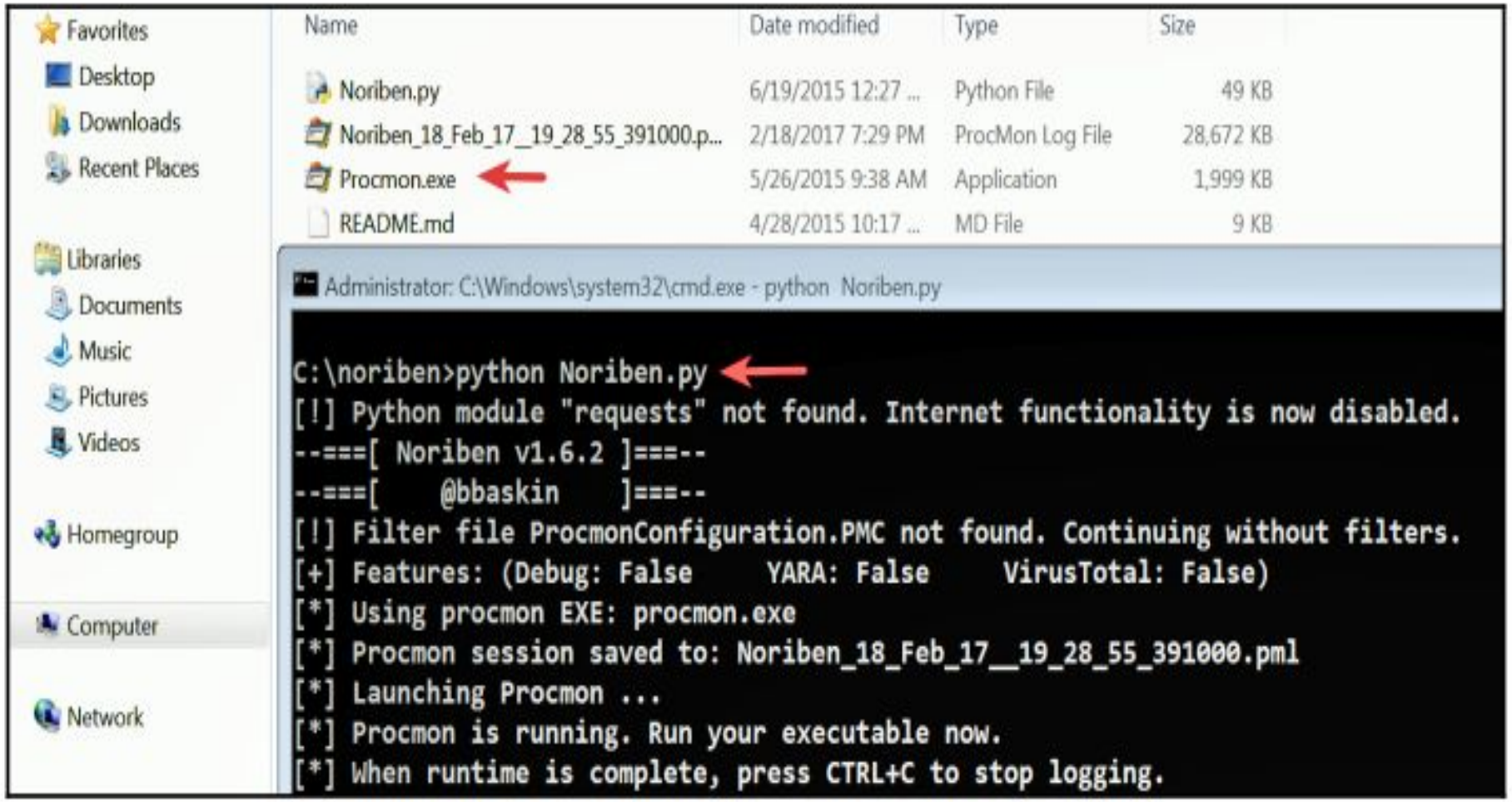


Dynamic Analysis

Logging System Activities using Noriben:-

- ✔ Noriben (https://github.com/Rurik/Noriben) is a Python script that works in conjunction with Process Monitor and helps in collecting, analyzing, and reporting runtime indicators of the malware.
- ✔ The advantage of using Noriben is that it comes with pre-defined filters that assist in reducing noise and allow you to focus on the malware-related events.
- ✔ To use Noriben, download it to your Windows VM, extract it to a folder, and copy Process Monitor (Procmon.exe) into the same folder before running the Noriben.py Python script, as shown in the following screenshot:

Logging System Activities using Noriben:-



Logging System Activities using Noriben:-

When you run *Noriben*, it launches *Process Monitor*. Once you are done with the monitoring, you can stop Noriben by pressing **Ctrl + C**, which will terminate Process Monitor. Once terminated, Noriben stores the results in a *text file* (`.txt`) and a *CSV file* (`.csv`) in the same directory. The *text file* contains events segregated based on the categories (like process, file, registry, and network activity) in separate sections, as shown in the following screenshot. Also, note that the number of events is much less because it applied predefined filters that reduced most of the unwanted noise:

Logging System Activities using Noriben:-

```
-=] Sandbox Analysis Report generated by Noriben v1.7.2
-=] Developed by Brian Baskin: brian @@ thebaskins.com  @bbaskin
-=] The latest release can be found at https://github.com/Rurik/Noriben

-=] Execution time: 28.87 seconds
-=] Processing time: 0.20 seconds
-=] Analysis time: 1.51 seconds

Processes Created:
==================
[CreateProcess] notepad++.exe:3884 > "%ProgramFiles%\Notepad++\updater\gup.exe -v7.32"  [Child PID: 3752]

File Activity:
==================

Registry Activity:
==================
[RegDeleteValue] notepad++.exe:3884 > HKCU\Software\Microsoft\Windows\CurrentVersion\Internet Settings\ZoneMap\ProxyBypass
[RegDeleteValue] notepad++.exe:3884 > HKCU\Software\Microsoft\Windows\CurrentVersion\Internet Settings\ZoneMap\IntranetName
[RegSetValue] notepad++.exe:3884 > HKCU\Software\Microsoft\Windows\CurrentVersion\Internet Settings\ZoneMap\UNCAsIntranet  =  0
[RegSetValue] notepad++.exe:3884 > HKCU\Software\Microsoft\Windows\CurrentVersion\Internet Settings\ZoneMap\AutoDetect  =  1
[RegDeleteValue] notepad++.exe:3884 > HKCU\Software\Microsoft\Windows\CurrentVersion\Internet Settings\ZoneMap\ProxyBypass
[RegDeleteValue] notepad++.exe:3884 > HKCU\Software\Microsoft\Windows\CurrentVersion\Internet Settings\ZoneMap\IntranetName
[RegSetValue] notepad++.exe:3884 > HKCU\Software\Microsoft\Windows\CurrentVersion\Internet Settings\ZoneMap\UNCAsIntranet  =  0
[RegSetValue] notepad++.exe:3884 > HKCU\Software\Microsoft\Windows\CurrentVersion\Internet Settings\ZoneMap\AutoDetect  =  1

Network Traffic:
==================
```

Logging System Activities using Noriben:-

The *CSV file* contains all the events (process, file, registry, and network activity) sorted by the timeline (the order in which the events occurred), as shown in the following screenshot:

Noriben_09_Jul_17__10_16_13_078000_timeline.csv

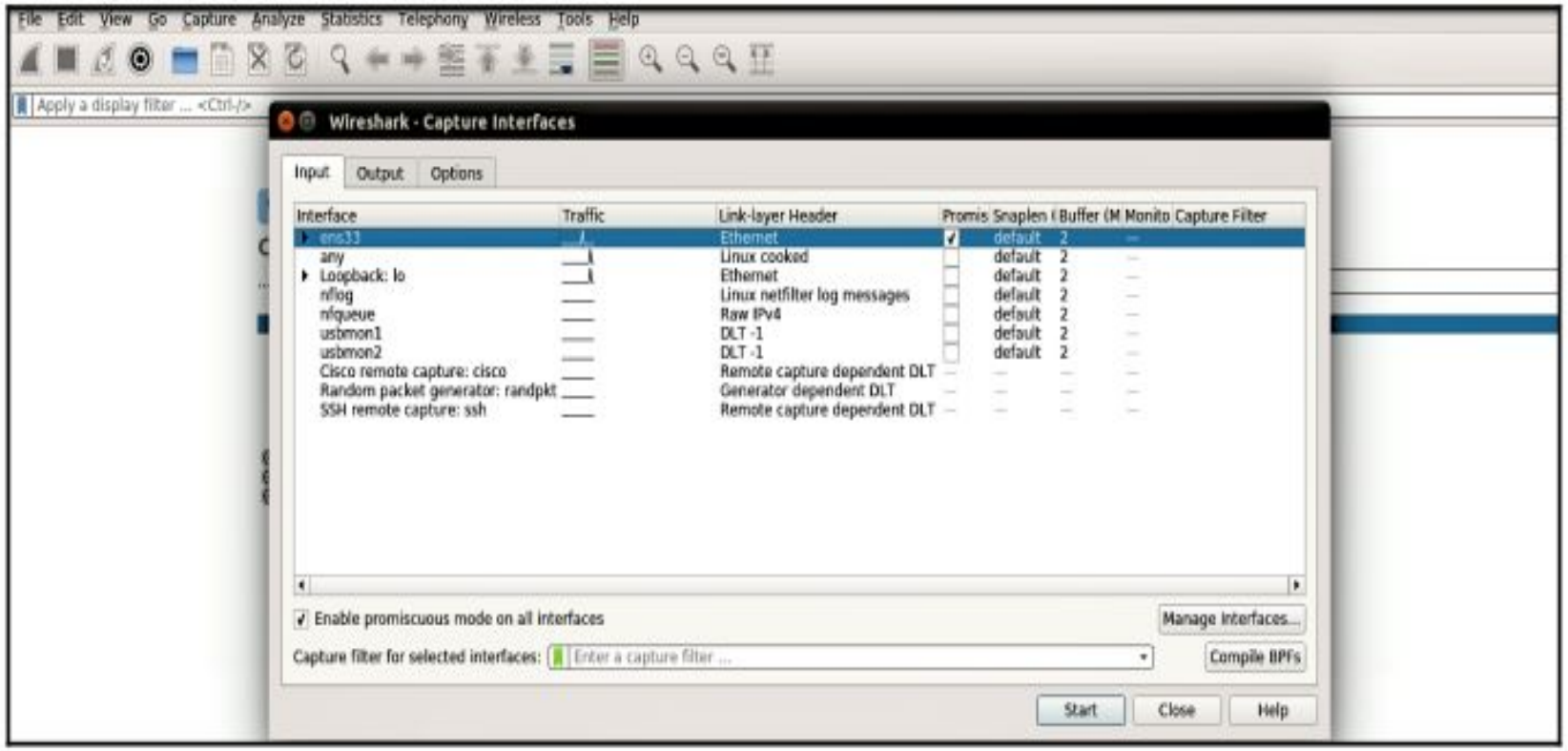
```
1 10:16:23,Registry,RegDeleteValue,notepad++.exe,3884,HKCU\Software\Microsoft\Windows\CurrentVersion\Internet Settings\ZoneMap\ProxyBypass
2 10:16:23,Registry,RegDeleteValue,notepad++.exe,3884,HKCU\Software\Microsoft\Windows\CurrentVersion\Internet Settings\ZoneMap\IntranetName
3 10:16:23,Registry,RegSetValue,notepad++.exe,3884,HKCU\Software\Microsoft\Windows\CurrentVersion\Internet Settings\ZoneMap\UNCAsIntranet, = 0
4 10:16:23,Registry,RegSetValue,notepad++.exe,3884,HKCU\Software\Microsoft\Windows\CurrentVersion\Internet Settings\ZoneMap\AutoDetect, = 1
5 10:16:23,Registry,RegDeleteValue,notepad++.exe,3884,HKCU\Software\Microsoft\Windows\CurrentVersion\Internet Settings\ZoneMap\ProxyBypass
6 10:16:23,Registry,RegDeleteValue,notepad++.exe,3884,HKCU\Software\Microsoft\Windows\CurrentVersion\Internet Settings\ZoneMap\IntranetName
7 10:16:23,Registry,RegSetValue,notepad++.exe,3884,HKCU\Software\Microsoft\Windows\CurrentVersion\Internet Settings\ZoneMap\UNCAsIntranet, = 0
8 10:16:23,Registry,RegSetValue,notepad++.exe,3884,HKCU\Software\Microsoft\Windows\CurrentVersion\Internet Settings\ZoneMap\AutoDetect, = 1
9 10:16:23,Process,CreateProcess,notepad++.exe,3884,%ProgramFiles%\Notepad++\updater\gup.exe -v7.32,3752
```

The *text file* and the *CSV* file can give different perspectives. If you are interested in the summary of events based on the category then you can look at the text file; if you are interested in the sequence of events in the order in which it occurred then you can view the *CSV* file.

Capturing the Network with Wireshark:

```
$ sudo wireshark
```

To start capturing the traffic on a network interface, click on **Capture | Options** (Or press **Ctrl + K**), select the network interface, and click on **Start**:



Dynamic Analysis Tools

Simulating Services with INetsim

- ✔ Most malware tries to reach its command-and-control server, which should be blocked during analysis.
- ✔ Malware needs network services to function properly, so analysts must simulate the internet instead of allowing real connections.
- ✔ INetSim is a free Linux-based tool that simulates standard internet services like DNS, HTTP, and HTTPS.
- ✔ INetSim also runs dummy services to handle connections on nonstandard ports, enabling malware to operate normally in a controlled environment.

Simulating Services with Inetsim:

Once INetSim is launched it simulates various services, as shown in the following output and it also runs a dummy service that handles connections directed at nonstandard ports

```
$ sudo inetsim
INetSim 1.2.6 (2016-08-29) by Matthias Eckert & Thomas Hungenberg
Using log directory: /var/log/inetsim/
Using data directory: /var/lib/inetsim/
Using report directory: /var/log/inetsim/report/
Using configuration file: /etc/inetsim/inetsim.conf
Parsing configuration file.
Configuration file parsed successfully.
=== INetSim main process started (PID 2758) ===
Session ID: 2758
Listening on: 192.168.1.100
Real Date/Time: 2017-07-09 20:56:44
Fake Date/Time: 2017-07-09 20:56:44 (Delta: 0 seconds)
```

Simulating Services with InetSim:

```
 Forking services...
  * irc_6667_tcp - started (PID 2770)
  * dns_53_tcp_udp - started (PID 2760)
  * time_37_udp - started (PID 2776)
  * time_37_tcp - started (PID 2775)
  * dummy_1_udp - started (PID 2788)
  * smtps_465_tcp - started (PID 2764)
  * dummy_1_tcp - started (PID 2787)
  * pop3s_995_tcp - started (PID 2766)
  * ftp_21_tcp - started (PID 2767)
  * smtp_25_tcp - started (PID 2763)
  * ftps_990_tcp - started (PID 2768)
  * pop3_110_tcp - started (PID 2765)
  [...............REMOVED.
..............]
  * http_80_tcp - started (PID 2761)
  * https_443_tcp - started (PID 2762)
 done.
Simulation running.
```

Dynamic Analysis Tools

Simulating Services with InetSim:

- ✔ Logging and response capabilities: INetSim can log all communications from malware and respond to HTTP/HTTPS requests, returning files based on their extensions.
- ✔ Providing dummy files: If malware requests a file (e.g., a .exe), INetSim can send a dummy file, allowing analysts to observe what the malware does with it.
- ✔ Safe observation of malware behavior: Using INetSim, malware can be executed in a controlled environment without contacting the real C2 server.
- ✔ Example with network traffic: Without INetSim, malware trying to reach a C2 server may fail (e.g., DNS resolution fails), as seen in Wireshark traffic showing "Port Unreachable" messages.

No.	Time	Source	Destination	Protocol	Length	Info
5	3.174453370	192.168.1.50	192.168.1.100	DNS	82	Standard query 0xdb99 A rnd009.googlepages.com
6	3.174473089	192.168.1.100	192.168.1.50	ICMP	110	Destination unreachable (Port unreachable)
7	3.175928441	192.168.1.50	192.168.1.100	DNS	82	Standard query 0x90ec A rnd009.googlepages.com
8	3.175942095	192.168.1.100	192.168.1.50	ICMP	110	Destination unreachable (Port unreachable)
9	3.176474369	192.168.1.50	192.168.1.100	DNS	82	Standard query 0x0ec8 A rnd009.googlepages.com
10	3.176482649	192.168.1.100	192.168.1.50	ICMP	110	Destination unreachable (Port unreachable)
11	3.178283604	192.168.1.50	192.168.1.100	DNS	82	Standard query 0x7190 A rnd009.googlepages.com
12	3.178291685	192.168.1.100	192.168.1.50	ICMP	110	Destination unreachable (Port unreachable)

Simulating Services with InetSim:

From the following screenshot, it can be seen that the malware first resolves the C2 domain, which is resolved to the Linux VM's IP address of 192.168.1.100. Once resolved, it then makes an HTTP communication to download a file (settings.ini)

No.	Time	Source	Destination	Protocol	Length	Info
5	14.687164101	192.168.1.50	192.168.1.100	DNS	82	Standard query 0xdb99 A rnd009.googlepages.com
6	14.741586271	192.168.1.100	192.168.1.50	DNS	98	Standard query response 0xdb99 A rnd009.googlepages.com A 192.168.1.100
7	14.744866993	192.168.1.50	192.168.1.100	TCP	66	49166 → 80 [SYN] Seq=0 Win=8192 Len=0 MSS=1460 WS=256 SACK_PERM=1
8	14.744944799	192.168.1.100	192.168.1.50	TCP	66	80 → 49166 [SYN, ACK] Seq=0 Ack=1 Win=29200 Len=0 MSS=1460 SACK_PERM=1 WS=1
9	14.747176177	192.168.1.50	192.168.1.100	TCP	60	49166 → 80 [ACK] Seq=1 Ack=1 Win=65536 Len=0
10	14.747225954	192.168.1.50	192.168.1.100	HTTP	158	GET /setting.ini HTTP/1.1
11	14.747243298	192.168.1.100	192.168.1.50	TCP	54	80 → 49166 [ACK] Seq=1 Ack=105 Win=29312 Len=0

Simulating Services with InetSim:

From the following screenshot, it can be seen that the HTTP response was given by the HTTP server simulated by INetSim. In this case, the User-Agent field in the HTTP request suggests that the standard browser did not initiate the communication and such an indicator can be used to create network signatures:

```
GET /setting.ini HTTP/1.1
User-Agent: AutoIt
Host: rnd009.googlepages.com
Cache-Control: no-cache

HTTP/1.1 200 OK
Date: Tue, 11 Jul 2017 05:18:16 GMT
Content-Length: 258
Content-Type: text/html
Connection: Close
Server: INetSim HTTP Server
```

A tool such as INetSim allows a security analyst to quickly determine the malware's behavior and capture its network traffic without having to manually configure all the services (such as DNS, HTTP and so on).

Dynamic Analysis Steps

Dynamic Analysis Steps During dynamic analysis (behavioral analysis), you will follow a sequence of steps to determine the functionality of the malware.

The following list outlines the steps involved in the dynamic analysis:

- ✔ **Reverting to the clean snapshot**: This includes reverting your virtual machines to a clean state.
- ✔ **Running the monitoring/dynamic analysis tools**: In this step, you will run the monitoring tools before executing the malware specimen. To get the most out of the monitoring tools covered in the previous section, you need to run them with administrator privileges.
- ✔ **Executing the malware specimen**: In this step, you will run the malware sample with administrator privileges.
- ✔ **Stopping the monitoring tools**: This involves terminating the monitoring tools after the malware binary is executed for a specified time.
- ✔ **Analyzing the results:** This involves collecting the data/reports from the monitoring tools and analyzing them to determine the malware's behavior and functionality

Static Analysis of the Sample

```
$ file sales.exe
sales.exe: PE32 executable (GUI) Intel 80386, for MS Windows

$ md5sum sales.exe
51d9e2993d203bd43a502a2b1e1193da sales.exe
```

The ASCII strings extracted from the binary using the *strings* utility contains references to a set of batch commands, which looks like a command to delete files. The strings also show a reference to a batch file (`_melt.bat`), which indicates that upon execution, the malware probably creates a batch (`.bat`) file and executes those batch commands. The strings also have references to the `RUN` registry key; this is interesting because most malware adds an entry in the `RUN` registry key to persist on the system after reboot:

```
!This program cannot be run in DOS mode.
Rich
.text
`.rdata
@.data
.rsrc
[....REMOVED....]
:over2
If not exist "
" GoTo over1
del "
GoTo over2
:over1
del "
_melt.bat
[....REMOVED....]
Software\Microsoft\Windows\CurrentVersion\Run
```

Static Analysis of the Sample

Examining the imports shows references to *file system*-and *registry*-related API calls, indicating the malware's ability to perform file system and registry operations, as highlighted in the following output. The presence of API calls `WinExec` and `ShellExecuteA`, suggest the malware's capability to invoke other programs (create a new process):

```
kernel32.dll
  [.....REMOVED......]
  SetFilePointer
  SizeofResource
  WinExec
  WriteFile
  lstrcatA
  lstrcmpiA
  lstrlenA
  CreateFileA
  CopyFileA
  LockResource
  CloseHandle

shell32.dll
  SHGetSpecialFolderLocation
  SHGetPathFromIDListA
  ShellExecuteA

advapi32.dll
  RegCreateKeyA
  RegSetValueExA
  RegCloseKey
```

Static Analysis of the Sample

-

Querying the hash value from the *VirusTotal* database shows 58 antivirus detections, and signature names suggest that we are probably dealing with a malware sample called *PoisonIvy*. To perform the hash search from VirusTotal, you need internet access, and if you want to use the VirusTotal public API, then you need an API key, which can be obtained by signing up for a VirusTotal account:

```
$ python vt_hash_query.py 51d9e2993d203bd43a502a2b1e1193da
Detections: 58/64
VirusTotal Results:
  Bkav ==> None
  MicroWorld-eScan ==> Backdoor.Generic.474970
  nProtect ==> Backdoor/W32.Poison.11776.CM
  CMC ==> Backdoor.Win32.Generic!O
  CAT-QuickHeal ==> Backdoor.Poisonivy.EX4
  ALYac ==> Backdoor.Generic.474970
  Malwarebytes ==> None
  Zillya ==> Dropper.Agent.Win32.242906
```

Static Analysis of the Sample

-

```
AegisLab ==> Backdoor.W32.Poison.deut!c
TheHacker ==> Backdoor/Poison.ddpk
K7GW ==> Backdoor ( 04c53c5b1 )
K7AntiVirus ==> Backdoor ( 04c53c5b1 )
Invincea ==> heuristic
Baidu ==> Win32.Trojan.WisdomEyes.16070401.9500.9998
Symantec ==> Trojan.Gen
TotalDefense ==> Win32/Poison.ZR!genus
TrendMicro-HouseCall ==> TROJ_GEN.R047C0PG617
Paloalto ==> generic.ml
ClamAV ==> Win.Trojan.Poison-1487
Kaspersky ==> Trojan.Win32.Agentb.jan
NANO-Antivirus ==> Trojan.Win32.Poison.dstuj
ViRobot ==> Backdoor.Win32.A.Poison.11776
[.................REMOVED.........................]
```

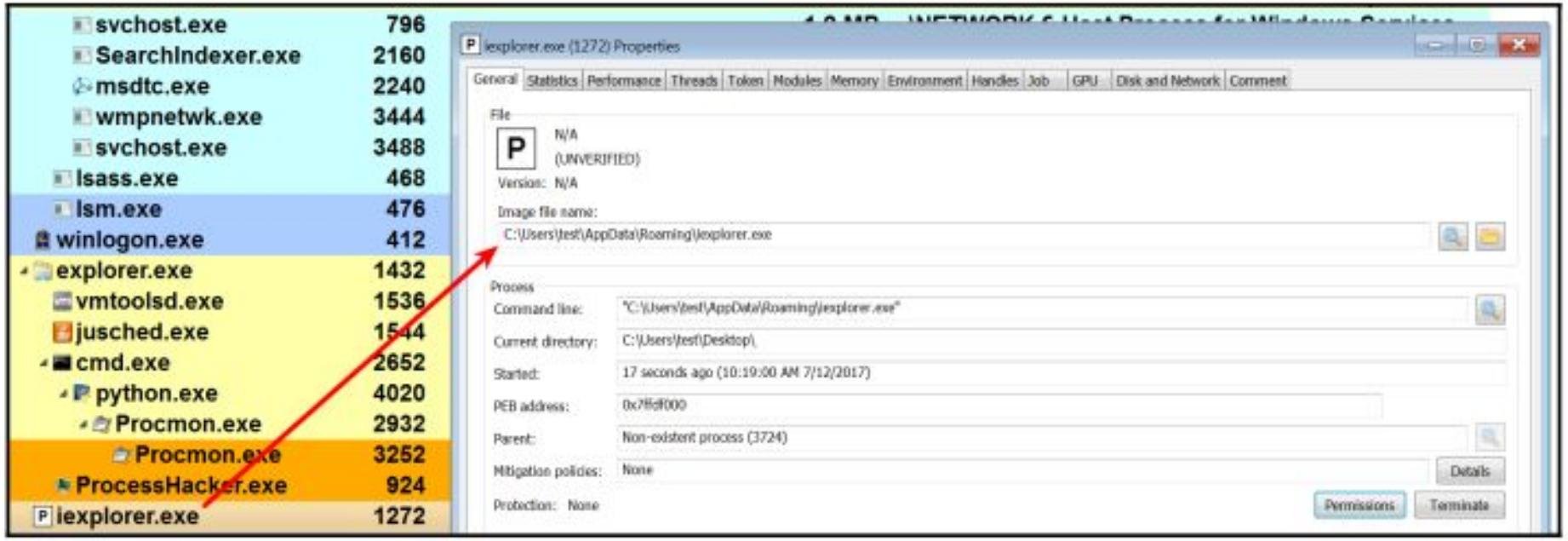
Dynamic Analysis of the Sample

1. Both the Windows VM and the Linux VM were reverted to the clean snapshots.
2. On Windows VM, *Process Hacker* was started with administrator privileges to determine process attributes, and the *Noriben* Python script was executed (which in turn started *Process Monitor*) to inspect the malware's interaction with the system.
3. On the Linux VM, *INetSim* was launched to simulate network services, and *Wireshark* was executed and configured to capture the network traffic on the network interface.
4. With all the monitoring tools running, the malware was executed with administrator privileges (right click | **Run as Administrator**) for around 40 seconds.
5. After 40 seconds, Noriben was stopped on the Windows VM. INetSim and Wireshark were stopped on the Linux VM.
6. Results from the monitoring tools were collected and examined to understand the malware's behavior.

Dynamic Analysis of the Sample

After performing dynamic analysis, the following information about the malware was determined from different monitoring tools:

1. Upon executing the malware sample (`sales.exe`), a new process, `iexplorer.exe`, was created with a process ID of `1272`. The process executable is located in the `%Appdata%` directory. The following screenshot is the output from *Process Hacker* showing the newly created process:



Dynamic Analysis of the Sample

2. By examining the Noriben logs, it can be determined that the malware dropped a file called `iexplorer.exe` in the `%AppData%` directory. The name of the file (`iexplorer.exe`) is similar to the file name of the Internet Explorer (`iexplore.exe`) browser. This technique is a deliberate attempt by the attacker to make the malicious binary look like a legitimate executable:

```
[CreateFile] sales.exe:3724 > %AppData%\iexplorer.exe
```

After dropping the file, the malware executed the dropped file. As a result of that, a new process `iexplorer.exe` was created. This was the process that was displayed by the *Process Hacker:*

```
[CreateProcess] sales.exe:3724 > "%AppData%\iexplorer.exe"
```

The malware then drops another file called `MDMF5A5.tmp_melt.bat`, as shown in the following output. At this point, it can be deduced that the `_melt.bat` string that we found during static analysis is concatenated with another string called `MDMF5A5.tmp`, which is used to generate a file name, `MDMF5A5.tmp_melt.bat`. Once the filename is generated, the malware drops a file with this name on the disk:

```
[CreateFile] sales.exe:3724 > %LocalAppData%\Temp\MDMF5A5.tmp_melt.bat
```

Dynamic Analysis of the Sample

It then executes the dropped batch (`.bat`) script by invoking cmd.exe:

```
[CreateProcess] sales.exe:3724 > "%WinDir%\system32\cmd.exe /c
%LocalAppData%\Temp\MDMF5A5.tmp_melt.bat"
```

As a result of `cmd.exe` executing the batch script, both the original file (`sales.exe`) and the batch script (`MDMF5A5.tmp_melt.bat`) were deleted, as shown in the following code snippet. This behavior confirms the delete functionality of the batch (`.bat`) file (if you recall, batch commands to delete files were found during the string extraction process):

```
[DeleteFile] cmd.exe:3800 > %UserProfile%\Desktop\sales.exe
[DeleteFile] cmd.exe:3800 > %LocalAppData%\Temp\MDMF5A5.tmp_melt.bat
```

The malicious binary then adds the path of the dropped file, as an entry in the `RUN` registry key for persistence, which allows the malware to start even after the system reboots:

```
[RegSetValue] iexplorer.exe:1272 >
HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Run\HKLM Key =
C:\Users\test\AppData\Roaming\iexplorer.exe
```

3. From the network traffic captured by *Wireshark*, it can be seen that the malware resolves the C2 domain and establishes a connection on port `80`:

7.637377173	192.168.1.50	192.168.1.100	DNS	… Standard query 0xf27d A www.webserver.proxydns.com
7.693976873	192.168.1.100	192.168.1.50	DNS	… Standard query response 0xf27d A www.webserver.proxydns.com A 192.168.1.100
7.865797192	192.168.1.50	192.168.1.100	DNS	… Standard query 0xf573 PTR 100.1.168.192.in-addr.arpa
7.883967058	192.168.1.100	192.168.1.50	DNS	… Standard query response 0xf573 PTR 100.1.168.192.in-addr.arpa PTR www.inetsim…
7.894688526	192.168.1.50	192.168.1.100	TCP	… 49173 → 80 [SYN] Seq=0 Win=8192 Len=0 MSS=1460 WS=256 SACK_PERM=1
7.894767035	192.168.1.100	192.168.1.50	TCP	… 80 → 49173 [SYN, ACK] Seq=0 Ack=1 Win=29200 Len=0 MSS=1460 SACK_PERM=1 WS=128
7.894902252	192.168.1.50	192.168.1.100	TCP	… 49173 → 80 [ACK] Seq=1 Ack=1 Win=65536 Len=0
7.894984480	192.168.1.50	192.168.1.100	TCP	… 49173 → 80 [PSH, ACK] Seq=1 Ack=1 Win=65536 Len=256
7.895002820	192.168.1.100	192.168.1.50	TCP	… 80 → 49173 [ACK] Seq=1 Ack=257 Win=30336 Len=0

Dynamic Analysis of the Sample

The TCP stream of the port 80 communication, as shown in the following screenshot, is not standard HTTP traffic; this suggests that the malware probably uses a custom protocol or encrypted communication. In most cases, the malware uses a custom protocol or encrypts its network traffic to bypass network-based signatures. You need to perform code analysis of malicious binaries to determine the nature of the network traffic. In the upcoming chapters, you will learn the techniques to perform code analysis in order to gain an insight into the inner workings of a malware binary:

Dynamic Analysis of the Sample

Comparing the cryptographic hash of the dropped sample (`iexplorer.exe`) and the original binary (`sales.exe`) shows that they are identical:

```
$ md5sum sales.exe iexplorer.exe
51d9e2993d203bd43a502a2b1e1193da sales.exe
51d9e2993d203bd43a502a2b1e1193da iexplorer.exe
```

To summarize, when malware is executed, it copies itself into the `%AppData%` directory as `iexplorer.exe` and then drops a batch script whose job is to delete the original binary and itself. The malware then adds an entry into the registry key so that it can start every time the system starts. The malicious binary possibly encrypts its network traffic and communicates with the *command and control (C2)* server on port 80 using a non-standard protocol.

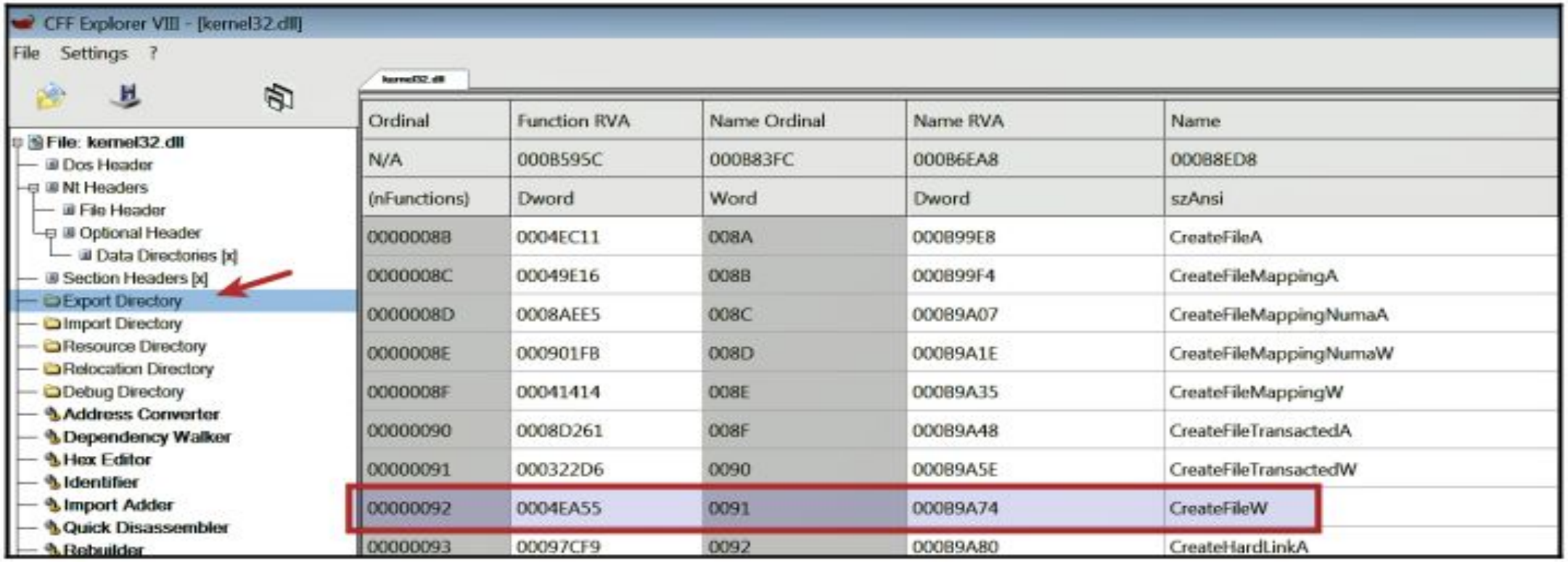
Dynamic Link Library (DLL)Analysis

A Dynamic-Link Library (DLL) is a module that contains functions (called exported functions or exports) that can be used by another program (such as an Executable or DLL). An executable can use the functions implemented in a DLL by importing it from the DLL.

The Windows operating system contains many DLLs that export various functions called Application Programming Interfaces (APIs). The functions contained in these DLLs are used by the processes to interact with the file system, process, registry, network, and the graphical user interface (GUI).

Dynamic Link Library (DLL)Analysis

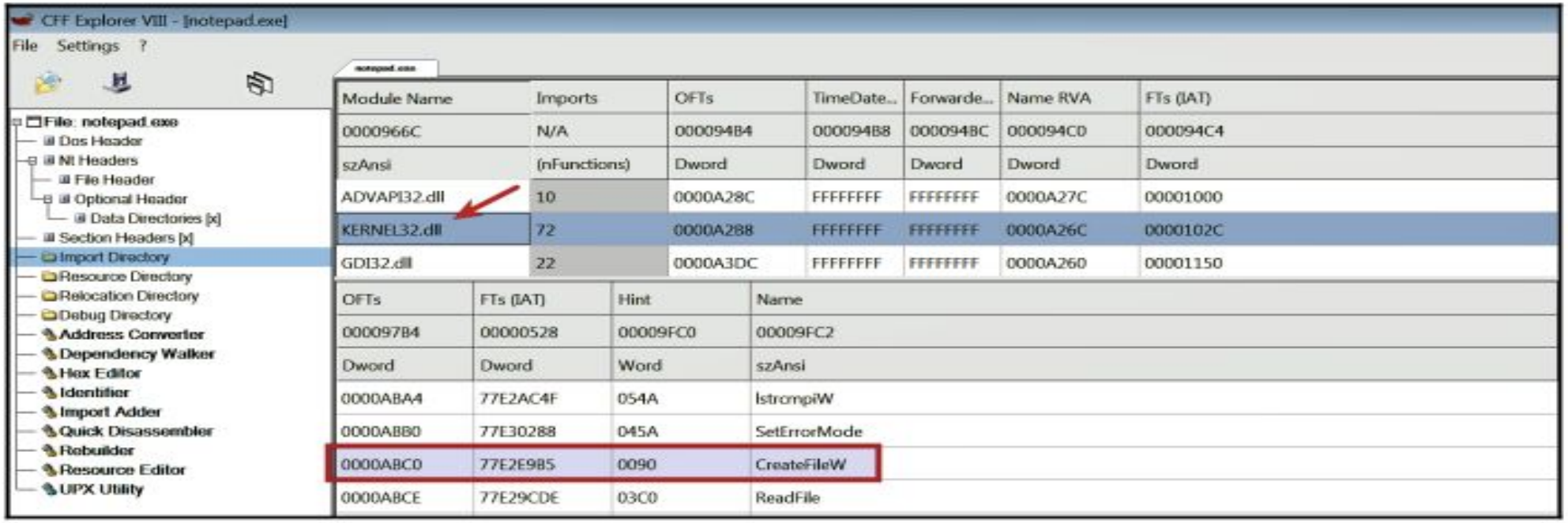
To display the exported functions in *CFF Explorer* tool, load the PE file that export functions and click on **Export Directory**. The following screenshot show some of the functions exported by `Kernel32.dll` (it is an operating system DLL and is located in the `C:\Windows\System32` directory). One of the functions exported by `Kernel32.dll` is `CreateFile`; this API function is used to create or open a file:



Ordinal	Function RVA	Name Ordinal	Name RVA	Name
N/A	000B595C	000B83FC	000B6EA8	00088ED8
(nFunctions)	Dword	Word	Dword	szAnsi
00000088	0004EC11	008A	000B99E8	CreateFileA
0000008C	00049E16	008B	000B99F4	CreateFileMappingA
0000008D	0008AEE5	008C	00089A07	CreateFileMappingNumaA
0000008E	000901FB	008D	00089A1E	CreateFileMappingNumaW
0000008F	00041414	008E	00089A35	CreateFileMappingW
00000090	0008D261	008F	00089A48	CreateFileTransactedA
00000091	000322D6	0090	00089A5E	CreateFileTransactedW
00000092	0004EA55	0091	00089A74	CreateFileW
00000093	00097CF9	0092	00089A80	CreateHardLinkA

Dynamic Link Library (DLL)Analysis

In the following screenshot, it can be seen that `notepad.exe` imports some of the functions exported by `kernel32.dll`, including the `CreateFile` function. When you open or create a file with Notepad, it calls the `CreateFile` API implemented in `Kernel32.dll`:



In the preceding example, notepad.exe did not have to implement the functionality to create or open the file in its code. To do that, it just imports and calls the CreateFile API implemented in Kernel32.dll.

The advantage of implementing the DLL is that its code can be shared by multiple applications. If an application wants to call an API function, it must first load a copy of DLL that exports the API into its memory space

Why attackers use DLL?

- ✔ A DLL cannot be executed by double-clicking; DLL needs a host process to run. By distributing the malicious code as a DLL, a malware author can load his/her DLL into any process, including a legitimate process such as Explorer.exe, winlogon.exe, and so on. This technique gives the attacker the capability to hide a malware's actions, and all the malicious activity performed by the malware will appear to originate from the host process.

- ✔ Injecting a DLL into an already running process provides the attacker with the capability to persist on the system.

- ✔ When a DLL is loaded by a process into its memory space, the DLL will have access to the entire process memory space, thereby giving it the ability to manipulate the process's functionality. For example, an attacker can inject a DLL into a browser process and steal credentials by redirecting its API function.

- ✔ Analyzing a DLL is not straightforward and can be tricky compared to analyzing an executable

Analyzing the DLL Using rundll32.exe

To determine the malware's behavior and to monitor its activity using dynamic analysis, it is essential to understand how to execute the DLL. As previously mentioned, a DLL needs a process to run. On Windows, rundll32.exe can be used to launch a DLL and to invoke functions exported from the DLL. The following is a syntax to launch a DLL and to invoke an export function using rundll32.exe:

rundll32.exe <full path to DLL>, <export function> <optional arguments>

The parameters associated with rundll32.exe are explained as follows:

- ✔ Full path to DLL: Specifies the full path to the DLL, and this path cannot contain spaces or special characters.
- ✔ Export function: This is a function in the DLL that will be called after the DLL is loaded.
- ✔ Optional arguments: The arguments are optional, and if supplied, these arguments will be passed to the export function when it is called.
- ✔ The comma: This is put between the full path to the DLL and the export function. The export function is required for the syntax to be correct.

Working of rundll32.exe

When we launch rundll32.exe, following steps are performed by rundll32.exe

1. Command-line arguments passed to rundll32.exe are first validated; if the syntax is incorrect, rundll32.exe terminates.

2. If the syntax is correct, it loads the supplied DLL. As a result of loading the DLL, the DLL entry point function gets executed (which in turn invokes the DLLMain function). Most malware implement their malicious code in the DLLMain function.

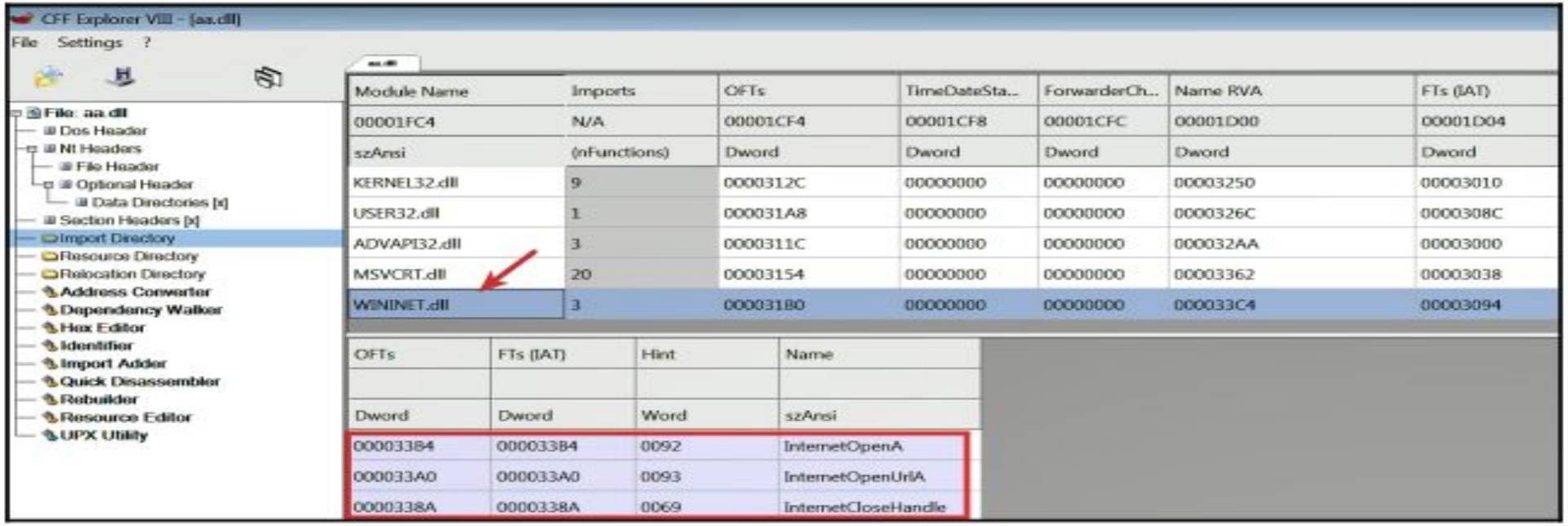
3. After loading the DLL, it obtains the address of the export function and calls the function. If the address of the function cannot be determined, then rundll32.exe terminates.

4. If the optional arguments are provided, then the optional arguments are supplied to the export function when calling it.

Launching the DLL Using rundll32.exe

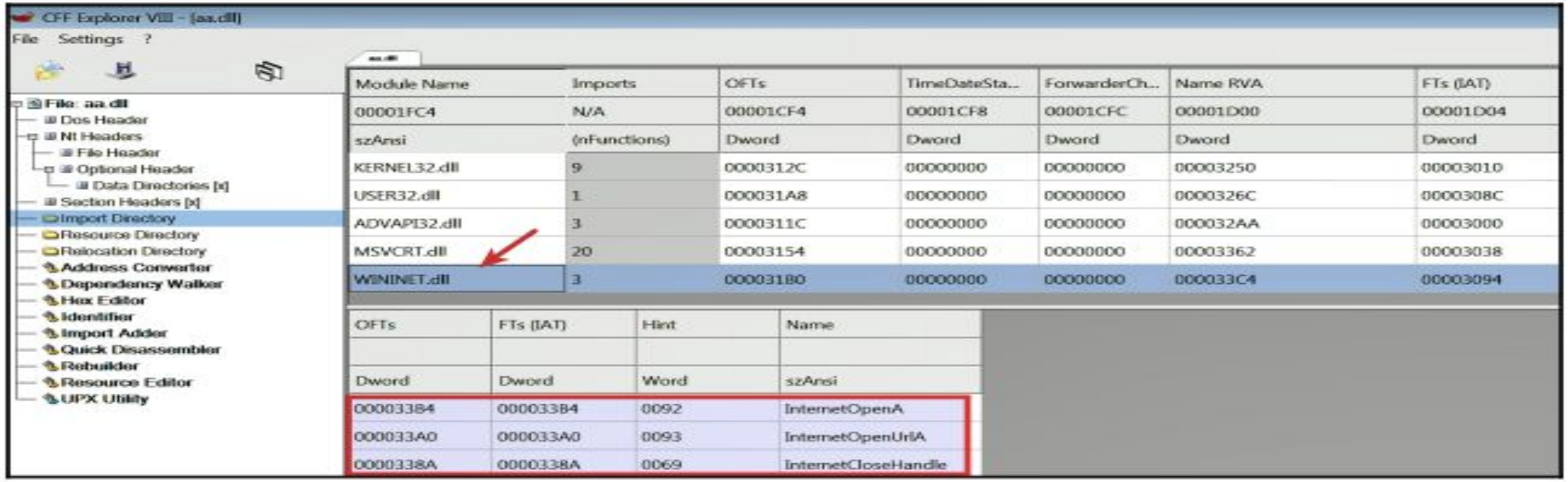
Example 1 – Analyzing a DLL With No Exports Whenever a DLL is loaded, its entry point function gets called (which in turn calls its DLLMain function). An attacker can implement malicious functionality (such as keylogging, information stealing, and so on) in the DLLMain function without exporting any functions.

In the following example, the malicious DLL (`aa.dll`) does not contain any exports, which tells you that, all the malicious functionality may be implemented in its `DLLmain` function, which will be executed (called from the `DLL entry point`) when the DLL gets loaded. From the following screenshot, it can be seen that the malware imports functions from `wininet.dll` (which exports the function related to HTTP or FTP). This indicates that the malware probably calls these network functions within the `DLLMain` function, to interact with the C2 server using HTTP or FTP protocol:



Launching the DLL Using rundll32.exe

In the following example, the malicious DLL (`aa.dll`) does not contain any exports, which tells you that, all the malicious functionality may be implemented in its `DLLmain` function, which will be executed (called from the `DLL entry point`) when the DLL gets loaded. From the following screenshot, it can be seen that the malware imports functions from `wininet.dll` (which exports the function related to HTTP or FTP). This indicates that the malware probably calls these network functions within the `DLLMain` function, to interact with the C2 server using HTTP or FTP protocol:



Launching the DLL Using rundll32.exe

You might assume that, because there is no export, a DLL can be executed using the following syntax:

```
C:\>rundll32.exe C:\samples\aa.dll
```

When you run a DLL with the preceding syntax, the DLL will not execute successfully; at the same time, you will not receive any error. The reason for this is that when `rundll32.exe` validates the command-line syntax (*step 1* mentioned in the *Section 6.2.1 Working of rundll32.exe*), it fails the syntax check. As a result, `rundll32.exe` exits without loading the DLL.

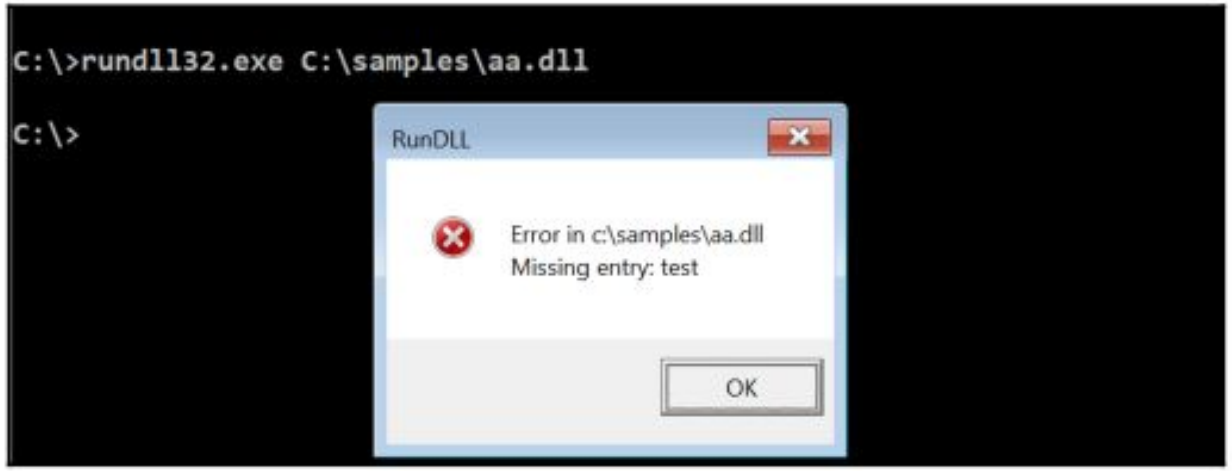
Launching the DLL Using rundll32.exe

You need to make sure that the command-line syntax is correct to load a DLL successfully. The command shown in the following output should run the DLL successfully. In the following command, `test` is a dummy name, and there is no such export function, it is just used to make sure the command-line syntax is correct. Before running the following command, the various monitoring tools that we covered in this chapter (Process Hacker, Noriben, Wireshark, Inetsim) were started:

```
C:\>rundll32.exe C:\samples\aa.dll,test
```

After running the command, the following error was received, but the DLL was successfully executed. In this case, because the syntax is correct, `rundll32.exe` loaded the DLL (*step 2*, mentioned in the *Section 6.2.1 Working of rundll32.exe*). As a result, its `DLL entry point` function was called (which in turn called `DLLMain`, containing the malicious code). Then `rundll32.exe` tries to find the address of the export function `test` (which is *step 3*, mentioned in the *Section 6.2.1 Working of rundll32.exe*). Since it cannot find the address of `test`, the following error was displayed. Even though the error message was displayed, the DLL was successfully loaded (that's exactly what we wanted for monitoring its activity):

Launching the DLL Using rundll32.exe



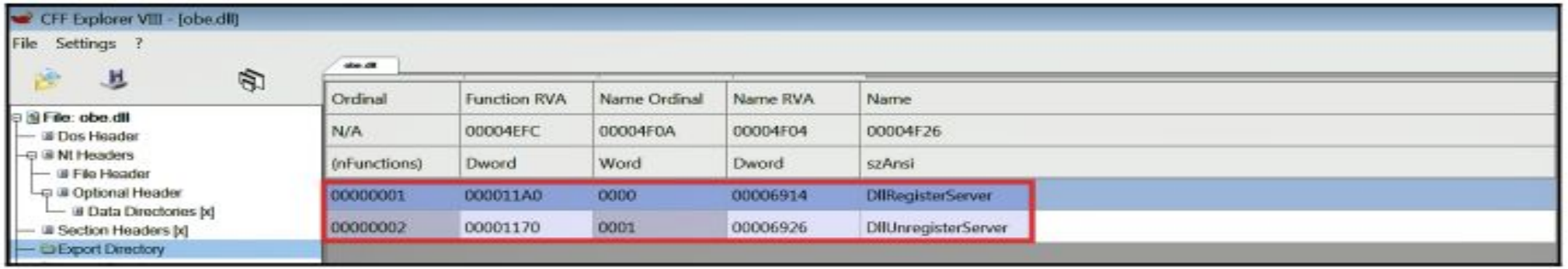
Upon execution, the malware establishes an HTTP connection with the C2 domain and downloads a file (`Thanksgiving.jpg`), as shown in the following *Wireshark* output:

No.	Time	Source	Destination	Protocol	Length	Info
	642.475022	192.168.1.50	192.168.1.100	DNS	76	Standard query 0xdb99 A www.giftnews.org
	742.480775	192.168.1.100	192.168.1.50	DNS	92	Standard query response 0xdb99 A www.giftnews.org A 192.168.1.100
	842.489943	192.168.1.50	192.168.1.100	TCP	66	49166 → 80 [SYN] Seq=0 Win=8192 Len=0 MSS=1460 WS=256 SACK_PERM=1
	942.489975	192.168.1.100	192.168.1.50	TCP	66	80 → 49166 [SYN, ACK] Seq=0 Ack=1 Win=29200 Len=0 MSS=1460 SACK_PERM=1 WS=128
	42.490120	192.168.1.50	192.168.1.100	TCP	60	49166 → 80 [ACK] Seq=1 Ack=1 Win=65536 Len=0
	42.490245	192.168.1.50	192.168.1.100	HTTP	226	GET /festival/Thanksgiving.jpg HTTP/1.1
	42.490252	192.168.1.100	192.168.1.50	TCP	54	80 → 49166 [ACK] Seq=1 Ack=173 Win=30336 Len=0

Launching the DLL Using rundll32.exe

Example 2 – Analyzing a DLL Containing Exports

In this example, we will look at another malicious DLL (`obe.dll`). The following screenshot shows two functions (`DllRegisterServer` and `DllUnRegisterServer`) exported by the DLL:



Ordinal	Function RVA	Name Ordinal	Name RVA	Name
N/A	00004EFC	00004F0A	00004F04	00004F26
(nFunctions)	Dword	Word	Dword	szAnsi
00000001	000011A0	0000	00006914	DllRegisterServer
00000002	00001170	0001	00006926	DllUnregisterServer

The DLL sample was run with the following command. Even though `obe.dll` was loaded into the memory of `rundll32.exe`, it did not trigger any behavior. This is because DLL's entry point function does not implement any functionality:

```
C:\>rundll32.exe c:\samples\obe.dll,test
```

On the other hand, running the sample with the `DllRegisterServer` function as shown below, triggered an HTTPS communication to the C2 server. From this, it can be deduced that `DLLRegisterServer` implements the network functionality:

```
C:\>rundll32.exe c:\samples\obe.dll,DllRegisterServer
```

Launching the DLL Using rundll32.exe

The following screenshot shows the network traffic captured by Wireshark:

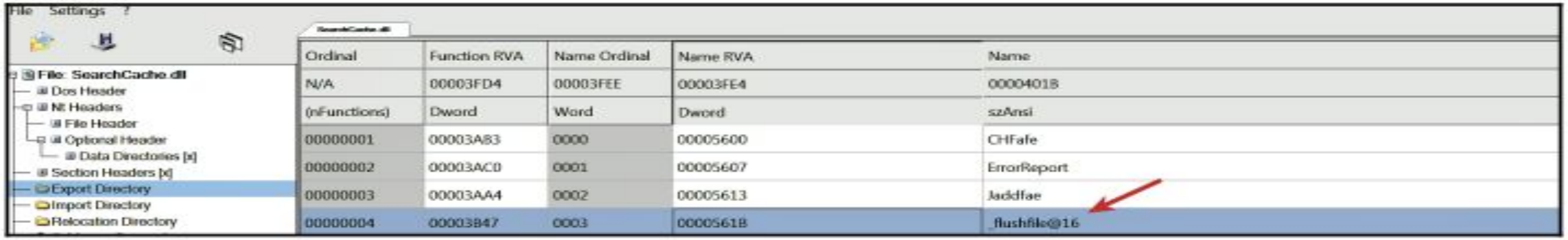
556.677039135	192.168.1.50	192.168.1.100	DNS	74	Standard query 0xa207 A inocnation.com
656.713504929	192.168.1.100	192.168.1.50	DNS	90	Standard query response 0xa207 A inocnation.com A 192.168.1.100
756.716057362	192.168.1.50	192.168.1.100	TCP	66	49166 → 443 [SYN] Seq=0 Win=8192 Len=0 MSS=1460 WS=256 SACK_PERM=1
856.716088408	192.168.1.100	192.168.1.50	TCP	66	443 → 49166 [SYN, ACK] Seq=0 Ack=1 Win=29200 Len=0 MSS=1460 SACK_PERM=1 WS=…
956.716266092	192.168.1.50	192.168.1.100	TCP	60	49166 → 443 [ACK] Seq=1 Ack=1 Win=65536 Len=0
56.717887835	192.168.1.50	192.168.1.100	TLSv1	176	Client Hello
56.717897210	192.168.1.100	192.168.1.50	TCP	54	443 → 49166 [ACK] Seq=1 Ack=123 Win=29312 Len=0
56.721129298	192.168.1.100	192.168.1.50	TLSv1	1359	Server Hello, Certificate, Server Key Exchange, Server Hello Done
56.732013311	192.168.1.50	192.168.1.100	TLSv1	188	Client Key Exchange, Change Cipher Spec, Encrypted Handshake Message
56.732221314	192.168.1.100	192.168.1.50	TLSv1	113	Change Cipher Spec, Encrypted Handshake Message

Launching the DLL Using rundll32.exe

Example 3 – Analyzing a DLL Accepting Export Arguments

The following example shows how you can analyze a DLL that accepts export arguments. The DLL used in this example was delivered via powerpoint, as described in this link: `https://securingtomorrow.mcafee.com/mcafee-labs/threat-actors-use-encrypted-office-binary-format-evade-detection/`.

The DLL (`SearchCache.dll`) consists of an export function, `_flushfile@16`, whose functionality is to delete a file. This export function accepts an argument, which is the file to delete:



Ordinal	Function RVA	Name Ordinal	Name RVA	Name
N/A	00003FD4	00003FEE	00003FE4	00004018
(nFunctions)	Dword	Word	Dword	szAnsi
00000001	00003A83	0000	00005600	CHFafe
00000002	00003AC0	0001	00005607	ErrorReport
00000003	00003AA4	0002	00005613	Jaddfae
00000004	00003847	0003	0000561B	_flushfile@16

To demonstrate the delete functionality, a test file (`file_to_delete.txt`) was created, and the monitoring tools were launched. The test file was passed an argument to the export function `_flushfile@16` using the following command. After running the following command, the test file was deleted from the disk:

```
rundll32.exe c:\samples\SearchCache.dll,_flushfile@16
C:\samples\file_to_delete.txt
```

Launching the DLL Using rundll32.exe

The following is the output from the Noriben logs showing `rundll32.exe` deleting the file (`file_to_delete.txt`):

```
Processes Created:
[CreateProcess] cmd.exe:1100 > "rundll32.exe
c:\samples\SearchCache.dll,_flushfile@16 C:\samples\file_to_delete.txt"
[Child PID: 3348]

File Activity:
[DeleteFile] rundll32.exe:3348 > C:\samples\file_to_delete.txt
```

Analyzing DLL with process checks

In malware analysis, analysts often encounter malicious DLL files that cannot run independently like executable (.exe) files.

To analyze their behavior, analysts commonly use tools such as rundll32.exe to load and execute the DLL.

However, many advanced malware samples include process-aware checks that verify which process is loading the DLL.

If the DLL detects that it is running inside an analysis tool or unexpected process, it may terminate or hide its malicious functionality.

To overcome this limitation, analysts use techniques such as DLL injection, which forces a legitimate process to load the DLL so that its real behavior can be observed during analysis.

Analyzing DLL with process checks

Limitations of rundll32.exe for DLLs:
While rundll32.exe is commonly used to execute exported functions within a DLL, it is not a universal solution. Some DLLs are designed to run only when loaded by a specific host process (such as explorer.exe or iexplore.exe). These DLLs often perform process-name or environment checks during execution. If they detect that they are running under an unexpected process like rundll32.exe, they may alter their behavior, refuse to execute, or terminate immediately. This makes rundll32.exe ineffective for triggering the intended functionality of such DLLs.

Process-Aware and Anti-Analysis DLL Behavior
Malware DLLs frequently include anti-analysis or evasion techniques. One such technique is verifying the parent or host process before executing malicious logic. If the DLL determines that it is running in an analysis environment or under a generic loader (e.g., rundll32.exe), it may deliberately fail its initialization routine or suppress its payload. This behavior is intended to avoid detection, reverse engineering, or sandbox analysis.

Analyzing DLL with process checks

DLL Injection as an Alternative Execution Method

When a DLL cannot be executed directly due to these checks, injecting it into a legitimate target process becomes necessary. DLL injection involves forcing a running process to load a specified DLL into its address space. Since the DLL is now hosted inside the expected process (such as explorer.exe), its internal checks pass, and the entry point function (DllMain) is successfully executed. This technique is widely used by advanced malware to blend into normal system activity.

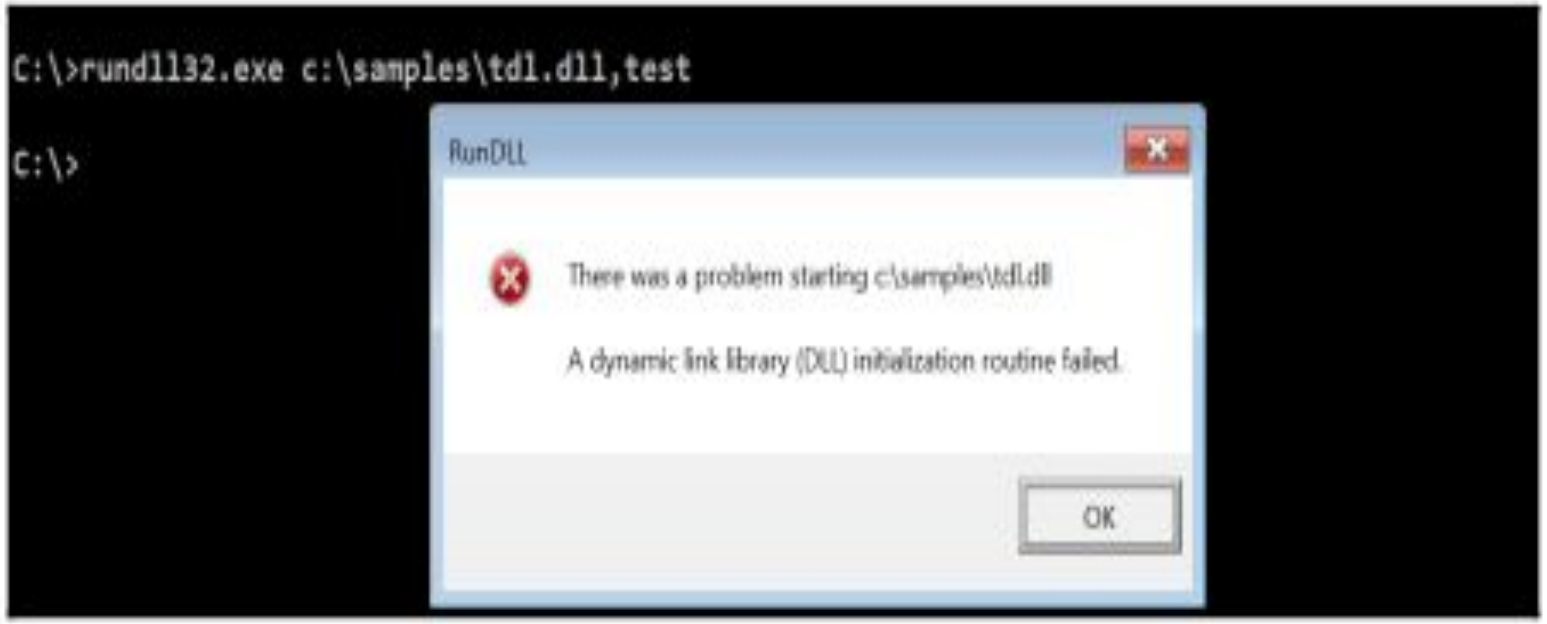
Use of RemoteDLL and Multiple Injection Techniques

Tools like RemoteDLL facilitate DLL injection into arbitrary processes and support multiple injection methods (e.g., CreateRemoteThread + LoadLibrary, APC injection, or manual mapping). Having multiple methods is important because certain processes may block or detect specific injection techniques due to security controls or privilege restrictions. If one method fails, another can be attempted, increasing the likelihood of successful injection and execution.

Analyzing DLL with process checks

Example of TDSS Rootkit DLL (tdl.dll)

The `tdl.dll` file from the TDSS rootkit is a practical example of such behavior. This DLL contains no exported functions, meaning it cannot be invoked via `rundll32.exe` in the usual way. Instead, all malicious logic resides in the DLL's entry point (`DllMain`). When executed directly, the system reports that the "DLL initialization routine failed," indicating that the entry point did not execute successfully. Injecting the DLL into the correct process allows the entry point to run as intended, thereby activating the rootkit's malicious functionality.



Analyzing DLL with process checks

To understand the condition that triggered the error, static code analysis (reverse engineering) was carried out. After analyzing the code, it was found that the DLL, in its entry point function, performed a check to determine if it is running under spoolsv.exe (the print spooler service). If it is running under any other process, the DLL initialization fails:

```
10001BF2   push    offset aSpoolsv_exe ; "spoolsv.exe"
10001BF7   push    edi                 ; char *
10001BF8   call    _stricmp
10001BFD   test    eax, eax
10001BFF   pop     ecx
10001C00   pop     ecx
10001C01   jnz     loc_10001CF9
```

Analyzing DLL with process checks

To trigger the behavior, malicious DLL had to be injected into the `spoolsv.exe` process using the *RemoteDLL* tool. After injecting the DLL into `spoolsv.exe`, the following activities were captured by the monitoring tools. The malware created a folder (`resycled`) and a file `autorun.inf` on the `C:\` drive. It then dropped a file `boot.com` in the newly created folder `C:\resycled`:

```
[CreateFile] spoolsv.exe:1340 > C:\autorun.inf
[CreateFolder] spoolsv.exe:1340 > C:\resycled
[CreateFile] spoolsv.exe:1340 > C:\resycled\boot.com
```

The malware added the following registry entries; from the added entries you can tell that the malware is storing some encrypted or configuration data in the registry:

```
[RegSetValue] spoolsv.exe:1340 > HKCR\extravideo\CLSID\(Default) =
{6BF52A52-394A-11D3-B153-00C04F79FAA6}
[RegSetValue] spoolsv.exe:1340 > HKCR\msqpdxvx\msqpdxpff = 8379
[RegSetValue] spoolsv.exe:1340 > HKCR\msqpdxvx\msqpdxaff = 3368
[RegSetValue] spoolsv.exe:1340 > HKCR\msqpdxvx\msqpdxinfo
=}gx~yc~dedomcyjloumllqYPbc
[RegSetValue] spoolsv.exe:1340 > HKCR\msqpdxvx\msqpdxid =
qfx|uagbhkmohgn""YQVSVW_,(+
[RegSetValue] spoolsv.exe:1340 > HKCR\msqpdxvx\msqpdxsrv = 1745024793
```

The following screenshot shows malware's C2 communication on port 80: